

주중 숙제 — 내 업무 미니앱 만들기 (프론트/서버 분리)

삼성창원병원 AI 업무지원 교육 · 다음 강의(다음 주 금요일) 전까지 각자 진행
· Claude Code로 복붙하며 만들기

🎯 이번 주 도달선: 내 PC에서 백엔드(서버)와 프론트(화면)가 따로 뜨고, 내 과제 질문 1개에 답이 나오면 성공. 욕심내지 말고 딱 여기까지. (다음 강의에서 확장)

🖥️ 먼저 — 내 환경 확인 (둘 중 하나) 이 핸드아웃의 프롬프트는 똑같이 쓰되, 파일 저장·실행 방법만 다릅니다.

(A) Claude Code (터미널/CLI 에이전트) — 프롬프트를 붙이면 파일 생성·터미널 실행·혹·Playwright까지 자동. 핸드아웃 그대로 따라가면 됩니다.

(B) 설치형 Claude 데스크톱 앱 (Windows 등) — Claude는 코드를 만들어 보여주지만 합니다. 그래서 본인이 직접: 1. Claude가 준 코드를 복사 → 메모장(또는 VS Code)에 붙여넣고 → 파일명 그대로 저장 (`app.py` , `index.html`). 한 폴더에 같이 두세요. 2. 아래 ‘실행’ 명령은 본인이 명령창(Windows = `cmd` 또는 `PowerShell`)에서 직접 칩니다. 3. `혹·Playwright`(심화)는 Claude Code 전용이라 데스크톱 앱에선 건너뛰니다. 핵심(분리 챗봇)만 해도 숙제 성공입니다.

🖥️ Windows 명령어: `python` 이 안 되면 `py` 로 바꿔 쓰세요. 예)
`py -m pip install fastapi uvicorn httpx / py -m uvicorn app:app --port 8000` (Python이 없으면 <https://www.python.org> 에서 설치 — 설치 시 “Add Python to PATH” 체크.)

1. 왜 “프론트 ↔ 서버”를 나누나? (지난 시간 교훈)

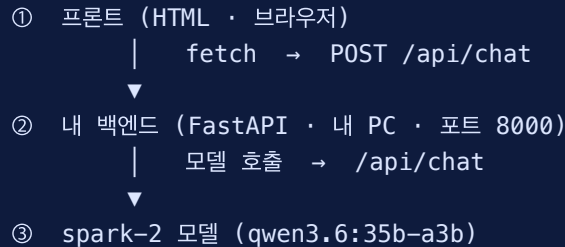
지난 시간 일부는 HTML 화면에서 자바스크립트로 AI 모델(ollama)을 직접 호출했습니다. 이러면:

❌ HTML → ollama 직접 호출의 문제 - CORS 오류로 일부 PC에서 아예 안 됨 - 모델 주소·키가 브라우저에 그대로 노출(보안) - 검색·규칙·출처 같은 로직을 넣을 곳이 없음 → 그냥 일반 챗

정답은 사이에 “내 백엔드(서버)”를 두는 것입니다:

✅ **프론트(HTML) → 내 백엔드(FastAPI) → spark-2 모델** - CORS 문제 없음(브라우저는 **내 백엔드**만 부름) - 자료·규칙·출처 처리를 백엔드에서 담당 - 나중에 검색·데이터베이스로 확장 쉬움

2. 공통 아키텍처



① 화면·질문 입력 → ② 자료·규칙·출처 처리(**내 컴퓨터에서 실행**) → ③ 모델만 원격(spark-2)

- 백엔드는 내 PC(로컬, 포트 8000)에서 돈다. spark-2에는 “모델”만 있고, 백엔드가 그 모델 API(**qwen3.6:35b-a3b**)만 가져다 쓴다.
- 이번 숙제는 **임베딩/검색 없이**, 질문과 함께 **참고 자료를 그대로 넘겨** (컨텍스트) 답하게 한다. (검색은 다음 강의)

3. 공통 복붙 프롬프트 (Claude Code에 순서대로)

아래 3개를 순서대로 Claude Code에 붙여넣으세요. 각자 과제 프롬프트 (개인별 쪽)는 이걸 끝낸 뒤에 엮습니다.

① 백엔드 만들기

```
FastAPI로 단일 파일 백엔드 app.py 를 만들어줘.
- 역할: 프론트가 보낸 {question, context} 를 받아,
spark-2 AI 모델에 보내 답을 돌려준다.
- 모델 호출: POST
http://121.138.151.6:11434/api/chat , 모델
"qwen3.6:35b-a3b" , stream:true
- 시스템 프롬프트: "참고 자료에만 근거해 한국어로 간결히 답한다.
자료에 없으면 '확인되지 않습니다'라고 한다."
- 엔드포인트: POST /api/chat (SSE 스트리밍으로 토큰 전송),
GET /health
- CORS 허용, 포트 8000, httpx 사용.
- 중요: 프론트(HTML)가 ollama를 직접 호출하지 않고 반드시 이
백엔드를 거치게 설계할 것.
```

② 프론트(화면) 만들기

단일 파일 index.html 프론트를 만들어줘.

- 구성: 질문 입력창 + 참고자료 입력칸 + 답변 영역 + 연결상태 표시등.
- 맨 위에 주소를 한 줄로 모아둘 것: `const BE = "http://127.0.0.1:8000";` (이후 모든 호출은 BE + `/api/chat` 처럼 이 변수만 사용)
- 중요: 호출은 오직 내 백엔드(BE)로만 한다. ollama(11434) 직접 호출 절대 금지.
- (개인 과제에서 새 엔드포인트를 추가하더라도, 화면은 별도 안내가 없으면 공통 ①의 `/api/chat(SSE)`를 그대로 호출한다.)
- POST로 `{question, context}` 전송, SSE 응답을 실시간으로 답변 영역에 표시.
- 시작 시 GET BE + `/health` 로 연결 확인 후 상태 표시.
- 한국어 UI, 깔끔하게.

③ 실행 (터미널)

```
pip install fastapi uvicorn httpx
# (먼저 포트가 비었는지 확인) - 아래가 응답하면 8000은 이미 사용 중:
curl -s http://127.0.0.1:8000/health
uvicorn app:app --port 8000          # ← 이 창은 켜둔 채로 둬(백엔드)
# 그 다음 index.html 을 더블클릭해 브라우저로 열기
```

△ 8000이 이미 사용 중이면(위 curl이 뭔가 응답하면) 다른 포트로 띄우세요: `uvicorn app:app --port 8001` — 이때 index.html 맨 위의 `const BE` 도 `:8001` 로 같이 바꿔야 화면이 연결됩니다(한 곳만 고치면 됨). 막히면 다음 강의 때 함께 봅니다. 여기까지 쓰면 숙제 성공.

4. 심화 ① — 훅(Hooks)으로 “실수 자동 차단” 에이전트

훅(Hook) = Claude Code에서 특정 순간(파일 수정 전후, 종료 시 등)에 자동으로 실행되는 명령. 설정 파일은 프로젝트의

`.claude/settings.json`.

가장 쓸모 있는 것: “HTML에서 ollama 직접 호출”을 쓰면 자동 차단하는 훅(이번 교훈 그대로).

훅은 이렇게 생겼어요 (`.claude/settings.json` — 작업 종료 시 `git status` 알림):

```
{ "hooks": { "Stop": [ { "matcher": "", "hooks": [
  { "type": "command", "command": "git status --short" }
] } ] } }
```

이런 JSON을 직접 외울 필요 없이, 아래 프롬프트로 Claude에게 만들게 하세요.

복붙 프롬프트:

```
내 프로젝트는 "HTML 프론트 → FastAPI 백엔드(:8000) → ollama(:11434)" 구조다.  
.claude/settings.json 에 넣을 혹은 JSON으로 만들어줘:  
1) PreToolUse: HTML 파일에 "11434" 또는 "ollama" 직접 호출이 들어가면 차단하고 이유를 알려줄 것  
2) PostToolUse: app.py(백엔드)가 수정되면 자동으로 점검 메시지 출력  
3) Stop: 작업 끝나면 git status 를 보여줄 것  
복붙 가능한 형식으로.
```

참고: [/hooks](#) 로 현재 훅 목록 확인, [/hookify](#) 로 대화 속 실수를 hook으로 자동 변환.

5. 심화 ② — Playwright 자동 QA 에이전트 에이전트

Playwright = 브라우저를 자동으로 조작해 내 앱을 테스트하는 도구. “사람이 매번 눌러보지 않아도, 에이전트가 대신 질문을 넣고 답이 뜨는지 검사”합니다.

복붙 프롬프트:

```
Playwright로 내 앱을 자동 테스트해줘.  
1) 백엔드(:8000) 연결 확인  
2) index.html 을 정적 서버 주소(http://127.0.0.1:5500)로 열기  
   ※ file:// 로 직접 열면 fetch가 CORS로 막힌다. 같은 폴더에서 python -m http.server 5500 을 따로 켜고 그 주소로 연다.  
3) 질문 입력칸에 "○○○(내 과제 질문)" 입력 후 전송  
4) 답변이 뜨고(모델 응답이 늦을 수 있으니 최대 30초 대기), 출처가 표시되는지 확인  
5) 통과/실패 리포트  
테스트 코드(tests/qa.spec.ts)와 실행 방법도 같이 만들어줘.
```

- 훅과 연계하면 **코드 바뀔 때마다 자동 QA**도 가능(위 4번 PostToolUse에서 호출).
- 설치 주의: 처음 1회 `npx playwright install` (브라우저 다운로드), Node 18+, 백엔드·모델이 먼저 떠 있어야 함. > 비개발자는 “Playwright로 테스트해줘”라고 Claude에 시키는 길부터. 어려우면 다음 강의에서.

6. 공통 안전수칙 (필독)

- 환자정보·대외비 자료 입력 금지 — 속제는 샘플·더미 조각으로만.
- spark-2 주소·접속법은 외부에 공유 금지(내부 교육 자료).
- 모델 답은 참고용 — 실제 적용은 담당 부서 확인.